

What does the work

Every QuickMail operation in the AI SDR Dashboard takes one of four paths. Which one is never a preference — it is dictated by what QuickMail exposes.

The rule. Use the documented API wherever an operation exists. Drop to QuickMail's internal endpoint only where the API has no equivalent at all. Drive the web UI only where neither API can do the thing. Each step down is more fragile and harder to run unattended, so nothing sits on a lower rung for convenience.

1

v2 GraphQL API

api.quickmail.com/v2/graphql · API key · no browser needed

Campaigns, leads, sequence steps, sending windows, mailbox list.

2

Internal GraphQL

next.quickmail.com/graphql · signed-in browser session · no password stored

Replies, sending a reply, stopping a prospect, both block lists, deliverability.

3

Playwright, clicking the UI

off by default · never throws into campaign creation

Two things only: the daily lead trigger, and unpausing a campaign.

4

Local only

never touches QuickMail

Suppression, reply classification, approvals, templates, campaign windows.

1 The documented API

Raw key in the `Authorization` header, 10 requests per 10 seconds. The only path that runs with no browser involved, so everything that can live here does.

Reads

<code>Campaigns</code>	names, paused, stats
<code>CampaignSteps</code>	positions, variation ids
<code>LeadsPage</code>	the lead list
<code>emailAccounts</code>	mailboxes, authorized, paused
<code>workspaces</code>	workspace id

Writes

<code>CreateCampaign</code>	the shell
<code>SetAccounts</code>	attach mailboxes
<code>CreateEmailStep</code>	the sequence
<code>CreateLeads</code> , <code>AddLeads</code>	enrol prospects
<code>UpdateAutomation</code>	days, hours, timezone
<code>SetPaused</code>	open/close the window

Its limits — which is the whole reason rungs 2 and 3 exist

- **Caps silently.** `leads(first:)` returns 10 however many you ask for. A pass over ~50,000 leads is ~4,800 requests, about 80 minutes at the rate limit — so the lead sync walks a slice per hour and carries a cursor.
`emailAccounts(first: 100)` also returns 10.
- **No company data.** The `Lead` type has no company field and no custom fields. Probed directly: `company` , `companyName` , `organization` , `customFields` , `attributes` , `website` , `industry` , `city` , `country` — all rejected.
- **No campaign pause, no trigger.** `paused` is writable on steps only.
- **Nothing about replies.** No message, thread or conversation type exists anywhere in the schema.

2 QuickMail's internal endpoint

The same endpoint QuickMail's own web app uses. Introspection is disabled, so every operation here was recovered by reading their JavaScript bundles — shapes copied from their call sites, never guessed. Their front end defines **326 mutations**; we use **seven**, and five queries.

How the session works. Requests are issued *inside* the signed-in page via `fetch('/graphql')`, so the cookie, CSRF token and origin all match. Playwright attaches to an already-running browser over CDP rather than logging in — **no QuickMail password is stored anywhere in this app.**

OPERATION	USED FOR
<code>appComponentsWorkspaceOpportunities</code>	Listing replies
<code>workspaceAppOpportunity</code>	One thread, with message bodies
<code>replyToEmail</code>	Sending a reply in-thread; also the manager forward
<code>setProspectsAsDoNotContact</code>	Marking a prospect do-not-contact
<code>cancelProspects</code>	Stopping their remaining sequence
<code>addDncEmails</code> / <code>removeDncEmails</code>	The blocked-address list
<code>addDncDomains</code> / <code>removeDncDomains</code>	The blocked-domain list
<code>workspaceDncEmailsPagination</code>	Reading both block lists
<code>accountinboxPagination</code>	Deliverability: <code>accredited</code> , <code>score</code> , <code>daily quota</code>

Why none of these can use the API

- **Replies.** The v2 schema has no reply, message or thread type. v1 REST was probed too — `/replies`, `/inbox`, `/messages`, `/threads`, `/conversations`, `/emails`, `/unibox`, `/activity`, `/events` — every one returns 404.
- **Block lists.** No v2 equivalent exists.
- **Deliverability.** v2 reports whether a mailbox is *connected*, never whether its mail *lands*. The `accredited` verdict exists only here — and it is what tells us a sender will go to spam.

Quirks that cost real debugging time

- **Two id namespaces.** v2 calls a mailbox `email_kKXM...`; this endpoint calls the same mailbox `198838`. Passing the v2 id returns "you don't have the permission to inbox".
- **searchFilter is a JSON string, and required.** Omitting it returns a 500 HTML error page, not a GraphQL error.
- **An invented field also returns 500 HTML**, naming nothing — which is exactly why shapes are copied from the bundle.
- **first is not obeyed.** Asking for 40 returns 30, so paging is explicit. Quiet truncation is how a sync starts looking complete when it isn't.
- **reply_type comes back null** on nearly every thread, so reply classification is ours to do.

3 Playwright, clicking the real UI

Used for exactly two things, both confirmed absent from both APIs first: **setting the daily trigger** (how many new leads start each day) and **unpausing the campaign itself**, as distinct from its steps.

The most fragile path in the system. It holds a real login rather than an API key, and QuickMail can break it at any time by changing their front end. Three rules follow: it is off unless `QUICKMAIL_UI_AUTOMATION=true`; it never throws into campaign creation, so a failure degrades to written manual instructions rather than losing a campaign that was created successfully; and every run is verified through the API afterwards.

v1 REST was checked as well before resorting to this: `/v1/accounts/{id}/campaigns` is a read-only list, and `/triggers`, `/automation`, `/pause`, `/resume`, `/start`, plus `PUT` and `PATCH` on campaigns, all 404.

The browser, and where it runs

The browser runs on the server, not on your machine. `connectOverCDP` dials `127.0.0.1`, which from the server's point of view is the server. Opening the dashboard from another computer needs no Edge, no Playwright and no QuickMail login — just a browser and an app password.

The server starts that browser itself when it isn't running, using a profile of its own with extensions disabled. One step still needs a person, once: signing in to QuickMail, because no password is stored. The profile keeps the session afterwards.

A consequence worth stating plainly: **the automation always acts as whichever QuickMail account that browser is signed in as.** It makes no difference who clicks, or from where.

4 What never touches QuickMail

AREA	WHY IT IS LOCAL
Suppression list	Keyed on the address, so re-importing a spreadsheet cannot resurrect someone who asked to stop. Mirrored <i>into</i> QuickMail's block list as well.
Reply classification	QuickMail returns <code>reply_type</code> null on nearly everything.
Approvals	The human-in-the-loop gate. Nothing there to sync.
Templates	Ours. QuickMail stores copy per step, not as a library.
Campaign windows	QuickMail has no start or end date, so the app enforces one by pausing and unpausing steps on a timer.
Managers, sync switches	App configuration.

What stops when the browser is unavailable

This is the practical difference between the rungs, and it is visible on the Sync tab every hour. Each part of the hourly pass is recorded independently, so a failed reply pull costs the reply pull and nothing else.

Still works

- Campaign stats
- Sending-mailbox list
- The lead walk
- Campaign start/end windows
- Everything local

Stops

- Pulling replies
- Sending a reply
- Stopping a prospect
- Both block lists
- Deliverability figures
- Forwarding to managers